UNIVERSIDADE FEDERAL DE ITAJUBÁ

Letícia Borges dos Reis - 2023005549
Maria Clara Rodrigues Ribeiro - 2023005772

Analisador Léxico e Sintático da Linguagem Mineirês

1º Trabalho de ECOM06A

Itajubá
2025

1. INTRODUÇÃO

A construção de compiladores representa uma das áreas fundamentais da computação, sendo essencial para a tradução de linguagens de programação de alto nível em instruções compreensíveis por máquinas. Este campo envolve diversas etapas interdependentes, que garantem desde a leitura e interpretação correta do código-fonte até sua conversão eficiente em código executável. Entre essas etapas, destacam-se a análise léxica e a análise sintática, componentes cruciais no processo de reconhecimento e validação da estrutura de programas escritos em linguagens formais.

Nesse sentido, o analisador léxico tem como função principal a leitura do código-fonte e a segmentação desse conteúdo em unidades atômicas chamadas tokens, que representam elementos como palavras-chave, identificadores, operadores e símbolos. Por outro lado, o analisador sintático atua sobre a sequência de tokens gerada, verificando se ela está de acordo com a gramática da linguagem, ou seja, com as regras que definem a estrutura válida das expressões e instruções. Juntas, essas duas fases garantem que o compilador consiga compreender corretamente a sintaxe de um programa antes de prosseguir para as etapas semânticas e de geração de código.

Neste contexto, o presente trabalho consistiu no desenvolvimento de um compilador voltado para a interpretação de uma nova linguagem de programação, criada apenas para fins didáticos e que foi baseada no dialeto mineiro, o mineirês. O projeto envolveu a definição das regras da linguagem, a construção do analisador léxico para identificação dos tokens e o desenvolvimento do analisador sintático baseado em uma gramática formal. O objetivo foi aplicar os conceitos fundamentais da teoria de compiladores por meio da criação de um sistema funcional e personalizado, capaz de processar instruções escritas em uma linguagem inspirada em modelos já consolidados, mas com adaptações criativas e culturais.

2. METODOLOGIA

Para o desenvolvimento do projeto, as atividades foram divididas em quatro fases: criação das regras da linguagem, definição dos tokens, criação das expressões regulares e regras de gramática e diagramação dos autômatos referentes às regras de produção.

Primeira fase: Essa fase consistiu em definir as regras da nova linguagem que seria lida pelo compilador. Nesse sentido, a linguagem desenvolvida para o trabalho foi uma linguagem de programação baseada principalmente em C, mas com referências à C++ e Python. Por outro lado, as principais modificações feitas foram que as palavras reservadas, em sua maioria, foram substituídas por expressões comuns do dialeto mineiro, o *mineirês*, e algumas estruturas foram alteradas conforme pode ser observado na tabela a seguir.

ESPECIFICIDADES DA LINGUAGEM
Semelhante à linguagem C com referências à C++ e Python
Delimitação de blocos é feita por chaves
Ponto e vírgula indica o fim do comando, não sendo necessário identar
Tem que ter ponto e vírgula no final de todos comandos, como se fosse tudo inline
Nome de variável só pode ser composto por letras
Não aceita declarar uma variável e atribuir valor a ela no mesmo comando
Não pode incrementar por ++ e decrementar por --
Não existe if else e não tem default no switch
Comentário é sempre em bloco (precisa abrir e fechar com ¬)
Cin e Cout tem _ em vez de >> e para imprimir variável é só colocar o id dela entre chaves (igual em python)

Segunda fase: Essa etapa consistiu na criação dos tokens de cada símbolo e palavra reservada para que o Analisador Léxico pudesse dividir o código em partes menores que representam alguma instrução. Tais definições podem ser encontrados na tabela da página 4.

Terceira fase: Nesse momento, foram construídas as expressões regulares e as regras de gramática que definirão como o Analisador Sintático verificará se a ordem dos tokens no código faz sentido. As expressões e regras criadas podem ser encontradas nas tabelas da página 5 e na página 6 está a explicação de cada uma individualmente.

Quarta fase: Por último, foram construídos os âutomatos das regras de gramática utilizando o software JFLAP. Os autômatos podem ser encontrados nas páginas 7 a 10.

3. RESULTADOS

Nas páginas a seguir estão as definições e construções relativas ao desenvolvimento do Analisador Léxico e Sintático do compilador. Em cada página, respectivamente, há:

1. Definição dos tokens de símbolos e palavras reservadas;
2. Expressões regulares e regras de produção;
3. Explicação de cada expressão regular e regra de produção;
4. Autômatos das regras de produção.

TOKENS					
Tipo	**Símbolo Original**	**Nosso Símbolo**	**Analisador Léxico**	**Token original**	**Nosso token**
Tipos de dados suportados	int	int	Palavra Reservada	T_INT	T_INT
	float	float	Palavra Reservada	T_FLOAT	T_FLOAT
	char	char	Palavra Reservada	T_CHAR	T_CHAR
Variável	letra	letra	Literais de Strings	T_ID	T_ID
	número	número	Literais Numéricos	T_NUM	T_NUM
Operadores relacionais	=	=	Operador de Atribuição	T_ATR	T_ATR
	==	==	Operador Relacional	T_EQ	T_IGUAL
	!=	!=	Operador Relacional	T_NEQ	T_DIF
	>=	>=	Operador Relacional	T_GEQ	T_MAIG
	<=	<=	Operador Relacional	T_LEQ	T_MEIG
	>	>	Operador Relacional	T_GT	T_MAIOR
	<	<	Operador Relacional	T_LT	T_MENOR
Operadores lógicos	&	&	Operador Relacional	T_AND	T_AND
	|	|	Operador Relacional	T_OR	T_OR
	~	~	Operador Relacional	T_NOT	T_NOT
Operadores Aritméticos	+	+	Operador Aritmético	T_PLUS	T_MAIS
	-	-	Operador Aritmético	T_MINUS	T_MENOS
	*	*	Operador Aritmético	T_MUL	T_VEZES
	/	/	Operador Aritmético	T_DIV	T_DIV
	mod	mod	Operador Aritmético	T_MOD	T_MOD
	e	e	Operador Aritmético	T_E	T_E
Símbolos especiais	.	.	Delimitadores	T_DOT	T_PONTO
	,	,	Delimitadores	T_COMMA	T_VIRG
	:	:	Delimitadores	T_COLON	T_DOISPT
	;	;	Delimitadores	T_DEL	T_PTVIRGULA
	\'	\'	Delimitadores	T_SINGLE_QUOTE	T_ASPA
	"	"	Delimitadores	T_STR	T_ASPAS
	(	(	Delimitadores	T_LPAREN	T_ABPAR
	)	)	Delimitadores	T_RPAREN	T_FEPAR
	{	{	Delimitadores	T_LBRACE	T_ABCHAV
	}	}	Delimitadores	T_RBRACE	T_FECHAV
	#	#	Símbolo Especial	T_POUND	T_HASHTAG
	//		Símbolo Especial	T_COMMENT	
	/* */	¬	Símbolo Especial	T_BLOCK_COMMENT	T_BLOCOMENT
	>>	_	Símbolo Especial	T_SHIFT_RIGHT	T_UNDERLINE
	<<	_	Símbolo Especial	T_SHIFT_LEFT	T_UNDERLINE
Palavras reservadas	if	temBase	Palavra Reservada	T_IF	T_TEMBASE
	else	elso	Palavra Reservada	T_ELSE	T_ELSO
	for	faizai	Palavra Reservada	T_FOR	T_FAIZAI
	while	uaile	Palavra Reservada	T_WHILE	T_UAILE
	switch	oncoto	Palavra Reservada	T_SWITCH	T_ONCOTO
	case	trem	Palavra Reservada	T_CASE	T_TREM
	cin	xove	Palavra Reservada	T_CIN	T_XOVE
	cout	oiaso	Palavra Reservada	T_COUT	T_OIASO
	void	paia	Palavra Reservada	T_VOID	T_PAIA
	null	nu	Palavra Reservada	T_NULL	T_NU
	break	xispa	Palavra Reservada	T_BREAK	T_XISPA
	return	vorta	Palavra Reservada	T_RETURN	T_VORTA
	define	facavo	Palavra Reservada	T_DEFINE	T_FACAVO
	include	bota	Palavra Reservada	T_INCLUDE	T_BOTA

EXPRESSÕES REGULARES		
Nome		**Expressão**
num	→	[0-9]
T_NUM	→	(+|-)? num+ (\.num+)? (exp(sinal)? num+)?
T_ID	→	[A-Za-z]+
caracteres	→	T_NUM | T_ID
relacional	→	T_MENOR | T_MAIOR | T_DIF | T_MEIG | T_MAIG | T_IGUAL
aritmetico	→	T_MAIS | T_MENOS | T_VEZES | T_DIV | T_MOD
logico	→	T_AND | T_OR | T_NOT
opRelacional	→	caracteres relacional caracteres
opAritmetica	→	caracteres aritmetico caracteres
opLogica	→	caracteres logico caracteres
condicao	→	opRelacional | (opRelacional logico condicao)
parametros	→	T_NU | (caracteres (T_PTUVIRGLA caracteres)*)
tipos	→	T_INT | T_FLOAT | T_CHAR | T_PAIA
comandos	→	Condicional | Enquanto | Switch | For | Cin | Cout | CallFunção | Atribuição | DecVariável

REGRAS DE PRODUÇÃO		
Nome		**Regra**
Condicional	→	T_TEMBASE T_ABPAR condicao T_FEPAR T_ABCHAV comandos+ T_PTVIRGULA T_FECHAV (T_ELSO T_ABCHAV comandos+ T_PTVIRGULA T_FECHAV)?
Enquanto	→	T_UAILE T_ABPAR condicao+ T_FEPAR T_ABCHAV comandos+ T_PTVIRGULA T_FECHAV
Switch	→	T_ONCOTO T_ABPAR T_ID T_FEPAR T_ABCHAV (T_TREM T_ASPA caracteres T_ASPA T_DOISPT comandos+ T_PTVIRGULA T_XISPA T_PTVIRGULA)+ T_FECHAV
For	→	T_FAIZAI T_ABPAR atribuicao condicao T_PTVIRGULA opAritmetica T_FEPAR T_ABCHAV comandos+ T_PTVIRGULA T_FECHAV
Cin	→	T_XOVE T_UNDERLINE T_ASPAS T_ID T_ASPAS T_PTVIRGULA
Cout	→	T_OIASO T_UNDERLINE T_ASPAS (caracteres | (T_ABCHAV T_ID T_FECHAV))+ T_ASPAS T_PTVIRGULA
Define	→	T_HASHTAG T_FACAVO T_ID caracteres
Include	→	T_HASHTAG T_BOTA T_ID
DecVariável	→	tipos T_ID T_PTVIRGULA
Atribuição	→	T_ID T_ATR (caracteres | opAritmetica) T_PTVIRGULA
DecFunção	→	tipos T_ID T_ABPAR parametros T_FEPAR T_ABCHAV comandos+ T_VORTA (caracteres | T_NU) T_PTVIRGULA T_FECHAV
CallFunção	→	T_ID T_ABPAR parametros T_FEPAR T_PTVIRGULA
Comentário	→	T_BLOCOMENT caracteres T_BLOCOMENT

EXPLICAÇÃO DAS EXPRESSÕES REGULARES	
Nome	**Finalidade**
num	Conjunto de todos algarismos - 0 a 9
T_NUM	Identifica um número
T_ID	Identifica uma letra
caracteres	Identifica uma letra ou um número
relacional	Conjuntro de caracteres relacionais - comparação de caracteres
aritmetico	Conjuntro de caracteres artiméticos - soma, subtração, etc
logico	Conjuntro de caracteres lógicos - AND, OR e NOT - comparação de expressões
opRelacional	Comparação entre dois caractes utilizando operador relacional, retorna TRUE ou FALSE
opAritmetica	Comparação entre dois caractes utilizando operador aritmético, retorna o resultado da operação
opLogica	Comparação entre duas expressões utilizando operador lógico, retorna TRUE ou FALSE
condicao	Representa uma expressão que é composta por uma ou mais condições
parametros	Representa uma expressão que é composta por um ou mais parâmetros
tipos	Conjunto dos tipos de dados da linguagem - INT, CHAR e FLOAT
comandos	Conjuntos dos comandos básicos da linguagem - Switch, For, etc

EXPLICAÇÃO DAS REGRAS DE PRODUÇÃO		
Nome	**Finalidade**	**Exemplo de uso**

Condicional	Regras de produção para estrutura de condição	temBase (condicao){
		¬ operações caso condição seja atendida ¬
		};
		elsô {
		¬ operações caso condição não seja atendida ¬
		};

Enquanto	Regras de produção para estrutura de iterações a partir de uma condição	uaile (condicao){
		¬ operações ¬
		};

Switch	Executar diferentes operações baseando-se no valor de uma variável	oncoto(x) {
		trem 1:
		¬ operações ¬
		xispa;
		};

For	Regras de produção para estrutura de iterações a partir de uma contagem	faizai (T_ID = x; T_ID < x; T_ID + 1) {
		¬ operações ¬
		};

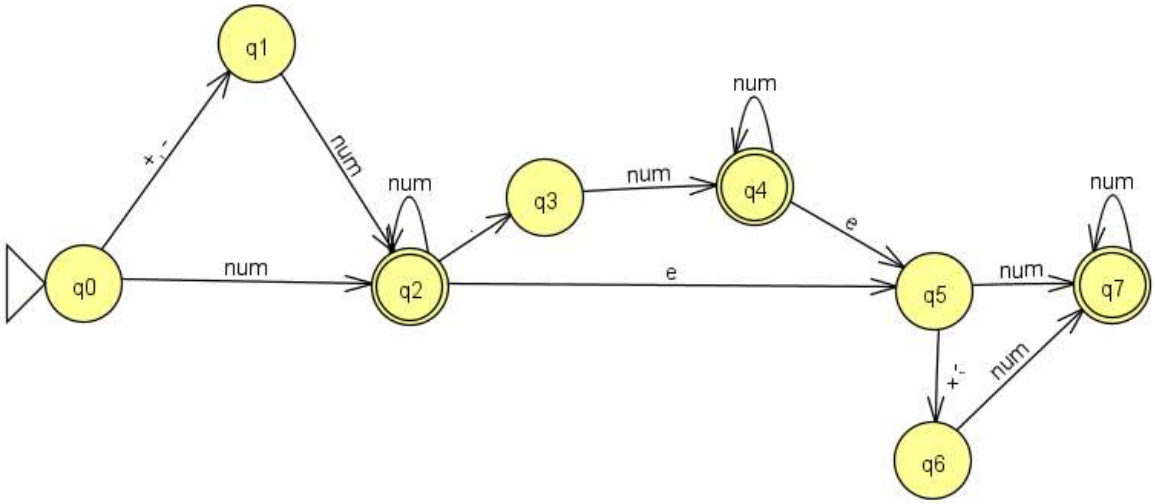
Cin / Cout	Comando para, respectivamente, ler entrada de dados e exibir dados na tela	xove_ x; oiaso_"valor = {x}";
Define	Define constantes (nome é substituído pelo valor no código em pré-compilação, serve apenas como abstração)	#facavo size 10
Include	Incluir arquivos no código, como bibliotecas	#bota NomeArquivo
Atribuição	Atribui um valor para uma variável, o valor atribuído pode ser um caracter ou o resultado de uma operação aritmética	T_ID = x; T_ID = x + y;
DecVariável	Criar uma variável	int x;

DecFunção	Criar a estrutura de uma função no código	tipo nomeFunc (parametros){
		¬ comandos ¬
		vorta valor;
		};

CallFunção	Executa a função referenciada passando os parâmetros necessários	nomeFunc (parametros);
Comentário	Anotações que são ignoras pelo compilador, podendo haver quebra de linha dentro de um bloco de comentário	¬ comentario ¬

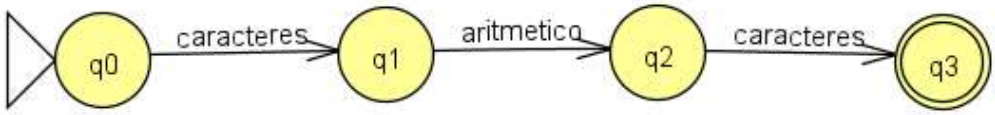
AUTÔMATOS

1) T_NUM

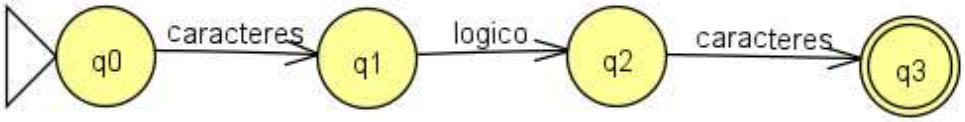


2) opRelacional

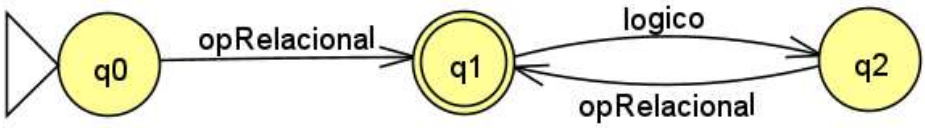
3) opAritmetica



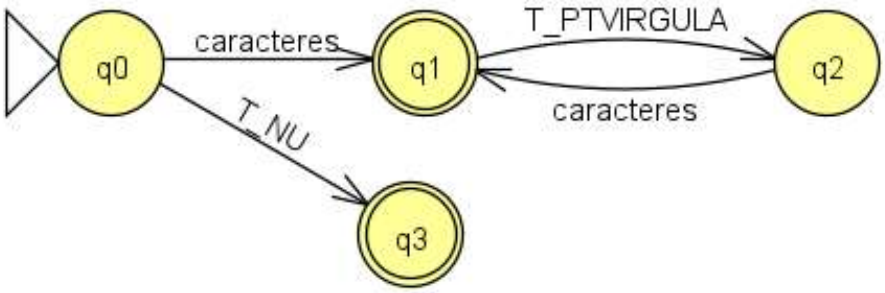
4) opLogica



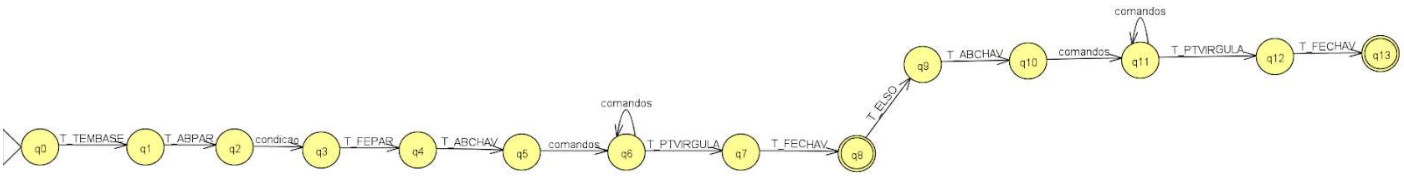
5) condicao



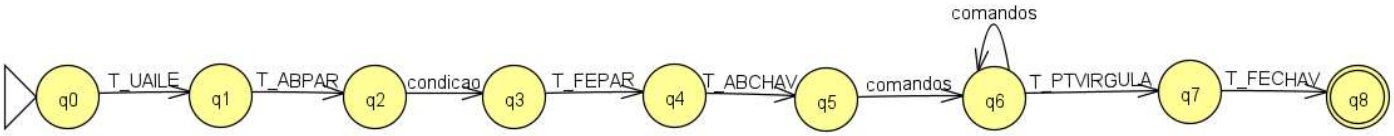
6) parametros



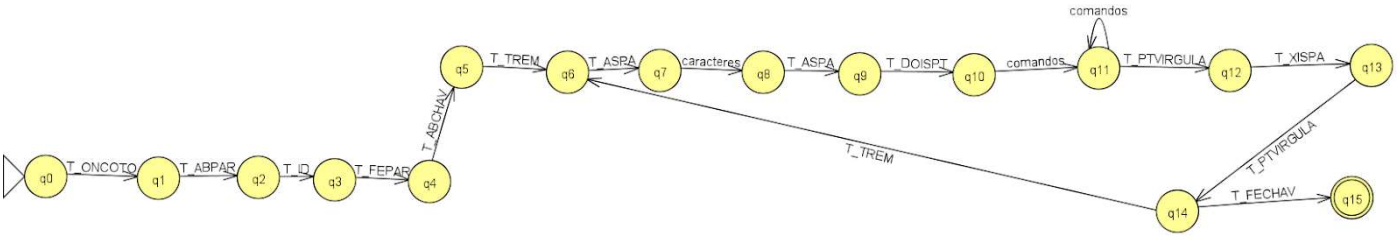
7) Condicional



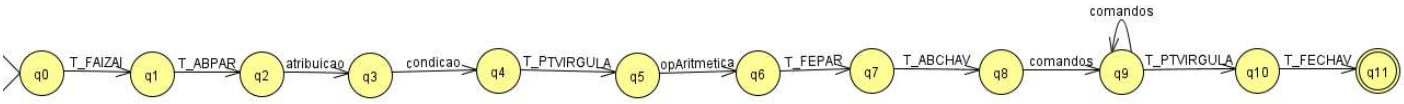
8) Enquanto



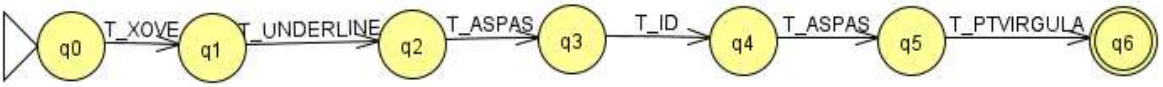
9) Switch



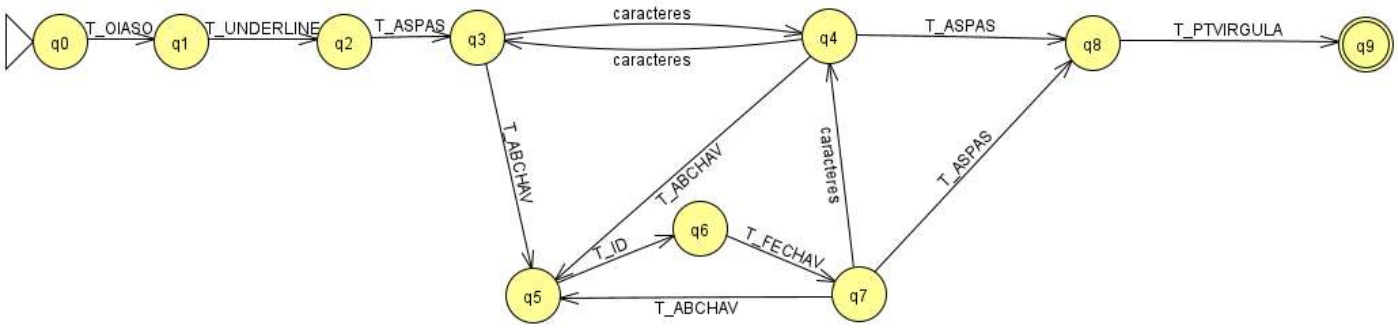
10) For



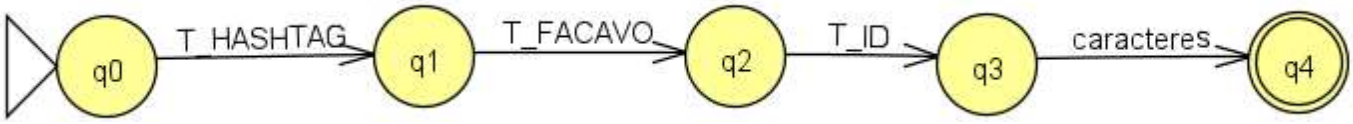
11) Cin



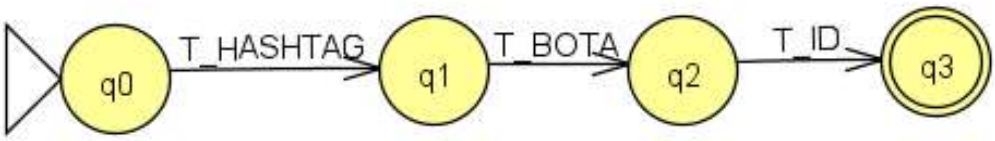
12) Cout



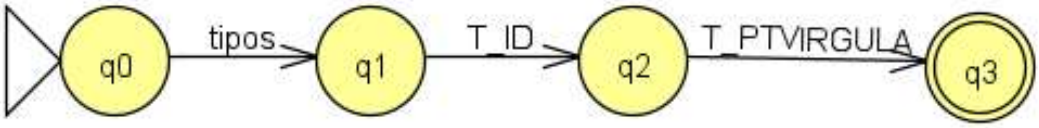
13) Define



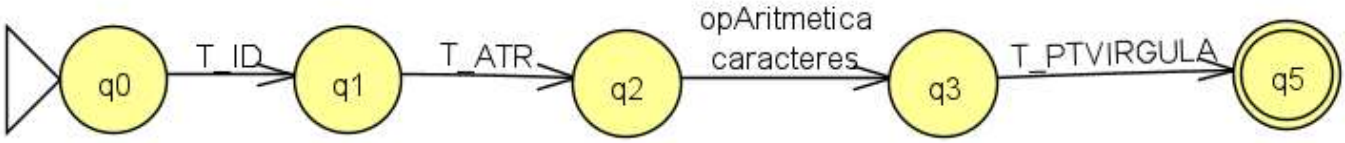
14) Include



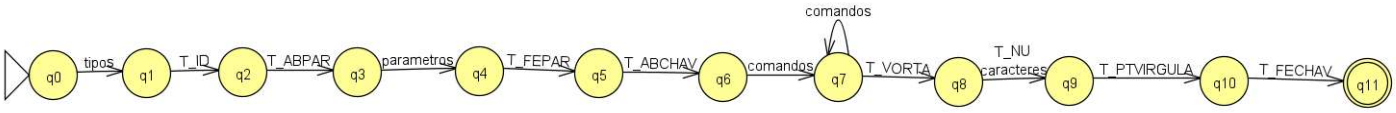
15) DecVariável



16) Atribuição

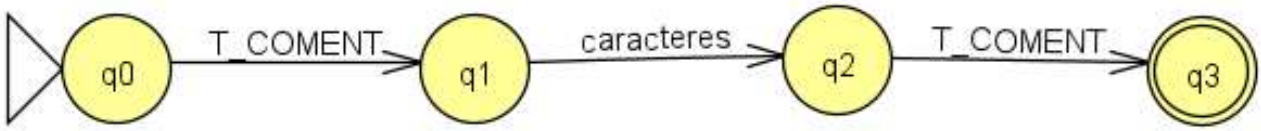


17) DecFunção



18) CallFunção

19) Comentário



4. CONCLUSÃO

A realização deste trabalho proporcionou uma compreensão aprofundada sobre o processo de desenvolvimento de um compilador, permitindo observar, na prática, a complexidade e a importância das etapas envolvidas na interpretação de linguagens de programação. Ao elaborar desde a definição da linguagem até a implementação dos analisadores léxico e sintático, foi possível consolidar conhecimentos teóricos adquiridos ao longo da formação acadêmica e compreender como essas ferramentas operam de forma integrada para garantir a correta leitura e execução de um código.

Além de reforçar o aprendizado técnico, o projeto evidenciou o papel central dos compiladores na área da computação. Nesse sentido, foi possível perceber, na prática, como eles são fundamentais para permitir a abstração entre linguagens de alto nível e o funcionamento interno das máquinas, possibilitando que desenvolvedores escrevam programas de forma mais legível, eficiente e independente da arquitetura subjacente. Dessa forma, o trabalho não apenas cumpriu seu objetivo didático, mas também ressaltou o valor dos compiladores como instrumentos essenciais para a construção de sistemas modernos e o avanço das tecnologias da informação.